To measure runtime of an algorithm

1. find $f(n)$ by counting the # of steps (primitive ops)
2. find the simplest $g(n)$ s.t $f(n) = O(g(n))$

Worst case running time Analysis

Example!

```
sum = 0                                          1
for j = 1 to n do                                c1 + n(c2 + n(c3 + 3n))
    for j = 1 to n do                            (c2 + n(c3 + 3n))
        for k = 1 to n do                        (c3 + 3n)
            sum = sum + 1                        3
return sum                                       ck
```

$$f(n) = 1 + c_1 + n\left(c_2 + n\left(c_3 + 3n\right)\right) + c_k$$

$$= 1 + c_1 + n c_2 + n^2 c_3 + 3n^3 + c_k$$

$$= 3n^3 + c_3 n^2 + c_2 n + (c_k + c_1 + 1)$$

$$= O(n^3)$$

Example

comparison op runs n times

while $n > 1$, do ← loop runs $(n-1)$ times

$n = (n-1)$ ← (3 ops)

return n ← c_1

$$f(n) = 1 + (n-1)(3) + c_1$$

$$= 3n - 2 + c_1$$

$$= O(n)$$

Example

while $n > 1$ do

$\log_2 n$ times

$n = n/2$ (3 ops)

return 0 $\leftarrow c_k$

$$f(n) = (\log_2 n)\,3 + c_k$$

$$= O(\log_2 n)$$

linearSearch$(A[1 \ldots n], x)$:

Input: an array $A[1 \ldots n]$ and an element x

Output: is x in the (possibly unsorted) array A?

```
1 for i := 1 to n:
2     if A[i] = x then
3         return True
4 return False
```

(a) Linear Search.

the worst possible input of size n,

$A = (2, 3, 7, 8)$ $x = 5$ ← would run the loop 4 times.

$A = (1, 6, 2, 5)$ $x = 6$ ← would run the loop 2 times.

$A = (7, 6, 5, 8)$ $x = 7$ ← would run the loop 1 time

↑ Best possible input of size n.

All of these arrays are of same length. However the # steps that we run are different.

Note: Even when the input size is n is the same, program may take different # of steps. So, run-time depends on not only input size but what the input is.

we could do

1. optimistic (best case)
2. pessimistic (worst case) → Easy to define; → guarantee for all inputs.
3. Neither (Average case)

↑ very appealing

- we need to know the distribution of the inputs.
- we need to make assumptions about the frequency of each input.

worst case running time analysis.

Intuitively: For each n (size of the input) what is the input that makes the running time worst?

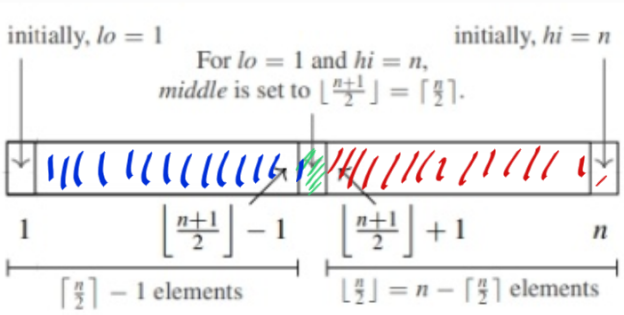
Def:

$$T(n) = \max_{x : |x| = n} [\text{the \# of primitive ops used by the algorithm on input } x]$$

```
binarySearch(A[1...n], x):
Input: a sorted array A[1...n]; an element x
Output: is x in the (sorted) array A?
1  lo := 1                                  } c1 ops
2  hi := n
3  while lo ≤ hi:
4      middle := ⌊(lo+hi)/2⌋
5      if A[middle] = x then
6          return True
7      else if A[middle] > x then
8          hi := middle − 1
9      else
10         lo := middle + 1
11 return False                             } c3 ops
```

c2 > 0

(b) The pseudocode for Binary Search.



(c) An illustration of the first split in binary search.

Best case input: x is in the middle of the array

Worst case input: x is not in the array A.

worst case analysis

claim: Worst case runtime of BS is $O(\log n)$

Proof: The worst case running time occurs when the input x is not in the array A, because while loop executes max # of iteration when x is not found.

Note that at each iteration, while loop halves the # of elements under consideration.

∴ # of times that we execute the loop for size n array A is $\log_2 n$.

overall, $f(n) = c_1 + c_2 \log_2 n + c_3$

$$f(n) = O(\log_2 n)$$

Analysis of recursive algorithms.

Problem: factorial

input: $n \in \mathbb{Z}^{\geq 0}$

output: $n! = n \cdot (n-1) \cdot (n-2) \cdots \cdot 1$

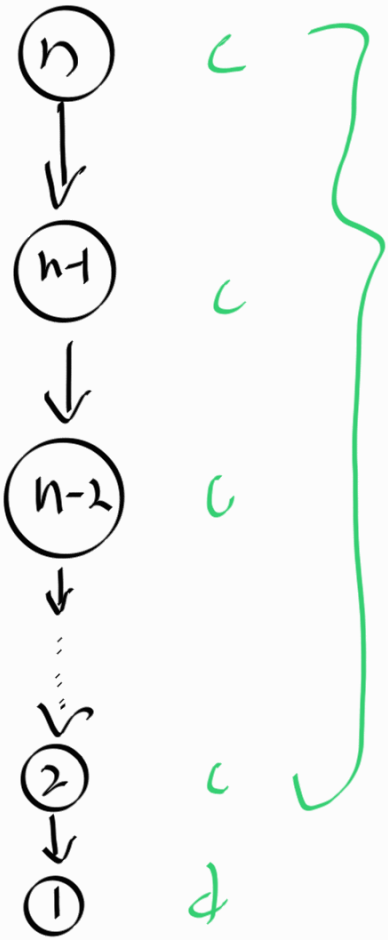
solution:

```
fact(n)
  if n==1 or n==0 then
      return 1
  else
      return n * fact(n-1)
```

What is the running time function?

Def : Recursion Tree

The recursion tree for a recursive algorithm A is a tree that shows all of the recursive calls spawned by a call to A on an input of size n. Each node in this tree is annotated with the amount of work aside from any other recursive calls, done by that call.




```
fact(n)
{ if n==1 or n==0 then
      return 1
{ else
      return n · fact(n-1)
```

$$f(n) = c(n-1) + d$$

$$= cn + d - c$$

$$= O(n)$$

Idea 2! using recurrence relation.

Def Recurrence relation

A recurrence relation (sometimes called a recurrence) is a function $T(n)$ that is defined (for some values of n) in terms of $T(k)$ for input values $k < n$.

```
fact(n)
  if n==1 or n==0 then
      return 1                    } d
  else
      return n * fact(n-1)        } c
```

Ex: for fact(n), the runtime $T(n)$ is

$$T(n) = \begin{cases} c + T(n-1) & n > 1 \\ d \end{cases}$$

But, we need a closed form solution for $T(n)$

non-recursive

(1) Iterate the solution a few times.

(2) Make a guess about closed form

(3) Try to prove guessed claim.

How? Induction.

$n > 1 : T(n) = c + T(n-1)$

$n = 1 : T(1) = d$

① $T(1) = d = d$

$T(2) = c + T(1) = c + d$

$T(3) = c + T(2)$

$= c + (c + d) = 2 \cdot c + d$

$T(4) = c + T(3)$

$= c + (c + (c + d)) = 3 \cdot c + d$

② $T(n) = (n-1) \cdot c + d$

③ use induction to prove

$T(n) = (n-1)c + d$.

Let's try to prove this.

$$T(n) = \begin{cases} d & n = 1 \\ T(n-1) + c & n > 1 \end{cases}$$

$P : \mathbb{N} \to \{T, F\}$, is defined as follows

$$P(n) = \begin{cases} \text{True}, & \text{If } T(n) = c(n-1)+d \\ \text{false}, & \text{otherwise.} \end{cases}$$

we are trying to prove

$[\forall n \geq 1 : P(n)]$

1. Base case is when $n=1$

When $n=1$, using recurrence relation $T(1) = d$.

and $T(1) = c \times (1-1) + d = d$

∴ The expression generates the correct value when $n \geq 1$

Base case is proven.

Inductive step:

$\forall n \geq 2 : P(n-1) \Longrightarrow P(n)$

Assume $P(n-1)$

$T(n-1) = (n-2)c + d \quad (IH)$

consider $T(n)$ using recurrence relation.

$$T(n) = T(n-1) + c$$

$$T(n) = (n-2)c + d + c \quad \text{(using IH)}$$

$$T(n) = (n-1)c + d$$

↑

This means $P(n)$ is true.

We have shown that

$$\forall n \geq 2 : P(n-1) \Rightarrow P(n)$$

∴ using principle of mathematical induction, we can conclude that

$$[\forall n \geq 1 : P(n)]$$

∴ $T(n) = c(n-1) + d$ for factorial algorithm.